Universidad Autonoma de Nuevo Leon

Facultad de Ingeniería Mecánica y Eléctrica

Nombre:	Matrícula:
Cepeda Macías Raúl Emilio	2145193
García Najera Eathan Omar	2144814
Contreras Elizondo Luis Adrián	2145018

Nombre del curso:	Nombre del profesor:
Componentes de lógica de negocios	Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	Actividad: Diseño Arquitectural del sistema

Fecha: 17/10/2026

Bibliografía (Formato APA)

Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.

Cisco Confidential

ACTIVIDAD PRACTICA

Diseño arquitectural del sistema con IA

Guía de trabajo para alumnos

En esta actividad convertirán el análisis de negocio elaborado en la Unidad 2 en una propuesta de arquitectura de software. Trabajarán con la misma empresa y el mismo caso. La inteligencia artificial podrá apoyar el diseño y la revisión, pero cada decisión deberá justificarse con evidencia del proyecto.

Empresa asignada	Hotel Costa Azul (Equipo 9)
Integrantes	Cepeda Macías Raúl Emilio 2145193 García Najera Eathan Omar 2144814 Contreras Elizondo Luis Adrián 2145018
Fecha de entrega	17 de Octubre de 2026

Cisco Confidential

1 Proposito de la actividad

El objetivo es organizar las reglas y responsabilidades del sistema antes de programarlo por completo. Al terminar, el equipo deberá demostrar cómo pasó de los requerimientos de la Unidad 2 a una arquitectura que separa el dominio, la aplicación y la infraestructura.

La actividad no consiste en pedir a una herramienta de IA que produzca una solución terminada. El trabajo consiste en plantear decisiones, solicitar alternativas, revisar las respuestas y documentar por qué una propuesta sí corresponde o no al negocio analizado.

Resultados de aprendizaje

- Comparar Transaction Script, Table Module y Domain Model con base en la complejidad real del caso.
- Modelar el dominio mediante lenguaje ubicuo, entidades, Value Objects, Aggregates y Domain Events.
- Diseñar una Service Layer que coordine casos de uso sin absorber las reglas del dominio.
- Separar la persistencia mediante Repository y definir el alcance transaccional con Unit of Work.
- Distinguir Commands y Queries en una propuesta básica de CQRS.
- Evaluar la arquitectura con los principios SOLID y registrar las decisiones en ADR.
- Usar IA como copiloto y conservar evidencia de la revisión humana.

2 Continuidad con la Unidad 2

Trabajarán con la misma empresa y no comenzarán un caso nuevo. Antes de diseñar la arquitectura, reúnan los productos elaborados en la Unidad 2 y asegúrense de que todos los integrantes comprenden el proceso principal del negocio.

Insumo de la Unidad 2	Como se utilizara en la Unidad 3
BRD	Delimitar alcance, actores, necesidades, restricciones, supuestos y criterios de exito.
Historias de usuario y Gherkin	Identificar casos de uso, condiciones de negocio y comportamientos esperados.
BPMN	Reconocer la secuencia, decisiones, participantes, excepciones y puntos de coordinacion.
Glosario y modelo de dominio	Mantener un lenguaje ubicuo y localizar conceptos con identidad, valor o ciclo de vida.
Matriz de trazabilidad	Relacionar cada decision arquitectural con una necesidad, historia, regla o escenario.
Repositorio de GitHub	Conservar versiones, diagramas, documentos, prompts y codigo generado o revisado.

Regla de continuidad

Si el equipo detecta una contradicción o un dato faltante en la Unidad 2, deberá registrarlo como hallazgo. Puede ajustar el análisis anterior, pero debe explicar el cambio y actualizar la trazabilidad. No se permite inventar nuevas reglas solo para acomodar una arquitectura.

Cisco Confidential

3 Empresas de trabajo

Cada equipo continuará con la empresa que trabajó en la Unidad 2. La siguiente tabla recuerda el contexto general de los nueve casos. Las preguntas sirven para enfocar la revisión; no indican qué patrón, clases o componentes deben elegir.

Empresa	Contexto	Pregunta de analisis
Mercado Norte	Alimentos, perecederos y promociones	¿Que reglas cambian por producto, sucursal, fecha o existencia?
TecnoNova Retail	Electronica, numeros de serie y garantias	¿Que decisiones requieren conservar identidad y ciclo de vida?
Moda Urbana	Ropa, tallas, colores y temporadas	¿Donde aparecen variantes, reservas, cambios y devoluciones?
HogarMax	Hogar, unidades voluminosas y ventas a credito	¿Que operaciones coordinan inventario, credito y entrega?
SaludPlus Farmacias	Medicamentos, lotes y caducidad	¿Que invariantes deben cumplirse antes de autorizar una operacion?
SportZone	Articulos deportivos y comisiones	¿Que reglas pertenecen a la venta y cuales a procesos posteriores?
Pet and Co	Productos, servicios y suscripciones	¿Que comportamientos dependen del plan, vigencia o tipo de servicio?
Casa and Estilo	Muebles, anticipos y entregas	¿Que estados y eventos cambian durante el cumplimiento de un pedido?
LibroMundo	Libros, ISBN y preventas	¿Que cambia entre una venta normal, una preventa y una cancelacion?

Cisco Confidential

4 Reglas generales

- Mantengan el alcance aprobado en el BRD y seleccionen un proceso principal para el diseño detallado.
- Todas las decisiones deben relacionarse con requerimientos, reglas, historias o escenarios existentes.
- No se evaluará la cantidad de clases. Se evaluará si cada responsabilidad está ubicada y justificada correctamente.
- Los nombres usados en diagramas, código y documentos deben coincidir con el glosario del negocio.
- El código puede ser parcial o boilerplate. Debe permitir revisar la organización y las responsabilidades propuestas.
- Conserven las iteraciones realizadas con IA. No entreguen una respuesta generada sin revisión.

5 Ruta de trabajo

Realicen las etapas en orden. Una etapa puede obligarlos a corregir una decisión anterior; registren el cambio en lugar de ocultarlo.

Etapa	Trabajo	Resultado
1	Preparar el caso	Revisar U2, delimitar proceso y seleccionar casos de uso.
2	Elegir el patron	Comparar alternativas y justificar la opcion elegida.
3	Construir el mapa de dominio	Definir lenguaje, responsabilidades, limites y eventos.
4	Diseñar la Service Layer	Organizar la coordinacion de los casos de uso.
5	Separar persistencia y transacciones	Proponer Repository y Unit of Work.
6	Separar comandos y consultas	Aplicar CQRS basico donde tenga sentido.
7	Revisar con SOLID	Detectar acoplamiento y responsabilidades incorrectas.
8	Representar la arquitectura	Crear UML, secuencias y ADR.
9	Validar y entregar	Trazar, probar, depurar y organizar evidencias.

Cisco Confidential

6 Etapa 1 Preparar el caso

Seleccionen de tres a cinco casos de uso relacionados con el proceso principal. Deben incluir al menos un flujo normal y un flujo con excepción o rechazo. Para cada caso de uso, identifiquen el actor, el objetivo, la información de entrada, las reglas involucradas y el resultado esperado.

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Consultar disponibilidad y aplicar tarifa por temporada	Reservaciones	BR-01, BR-02, BR-03, BR-04	El sistema muestra habitaciones compatibles y calcula automáticamente la tarifa (base o con incremento del 35% en temporada alta)
CU-02	Confirmar reserva con garantía o aplicar cancelación/no-show	Huésped / Reservaciones / Administración	BR-05, BR-06, BR-07	El sistema cambia la reserva a confirmada al registrar un anticipo del 50% o tarjeta, o bien aplica reembolsos/penalizaciones según los plazos (48 horas / 23:59 no-show).
CU-03	Realizar check-in, early check-in o late check-out	Recepción	BR-08, BR-09, BR-10, BR-11	El sistema registra datos de identidad (INE/Pasaporte), garantía incidental ($1,000 MXN o $500 MXN) y calcula cargos adicionales del 30% por early check-in o late check-out según la hora.
CU-04	Gestionar salida, consumos al folio, inspección y limpieza	Huésped / Recepción / Limpieza / Administración	BR-13, BR-14, BR-15, BR-16	Se consolidan los consumos al folio, se valida la inspección exprés de 5 minutos, se liquida la cuenta y se actualiza el estado de la habitación de 'Sucia' a 'Limpia/Inspeccionada' en tiempo real.

Antes de continuar, comprueben que los casos seleccionados aparecen en las historias de usuario o en el BPMN. Si no existe esa relación, el caso debe justificarse como corrección de alcance.

Cisco Confidential

7 Etapa 2 Elegir el patron de logica de negocio

Comparen Transaction Script, Table Module y Domain Model. La decisión deberá basarse en el comportamiento del sistema: cantidad de reglas, interacción entre conceptos, cambios esperados, manejo de estados y necesidad de proteger invariantes.

Trabajo solicitado

- Construyan una tabla comparativa con criterios propios del caso.
- Incluyan ventajas, riesgos y consecuencias de cada alternativa en su empresa.
- Seleccionen un patrón principal. Pueden proponer una combinación solo si delimitan dónde se usa cada patrón.
- Relacionen la decisión con al menos tres evidencias de la Unidad 2.

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Baja; dispersa la lógica en procedimientos o scripts individuales por cada pantalla o acción.	Media; agrupa la lógica orientada a tablas de base de datos, útil para reportes o operaciones tabulares simples.	**Alta y rica en comportamiento**; encapsula reglas complejas, cálculos y estados dentro de objetos de negocio interconectados.	BR-04, BR-06, BR-07, BR-11 (cálculos de penalizaciones por cancelación, no-show, y salidas anticipadas con reglas de tiempo estrictas).
Cambios esperados	Alto riesgo de duplicación de código si las políticas del hotel (ej. temporadas, recargos) cambian.	Moderado; facilita cambios si la estructura relacional es estable, pero acopla el negocio al diseño de BD	**Bajo riesgo y alta cohesión**; los cambios en políticas de negocio se encapsulan en las entidades y objetos de valor correspondientes	BR-02, BR-04, BR-10 (ajustes en tarifas automáticas por temporada alta/baja y políticas de cargos por horario)

Cisco Confidential

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Interaccion entre conceptos	Deficiente; las operaciones cruzan múltiples tablas mediante consultas manuales sin un modelo unificado.	Moderada; opera a nivel de conjuntos de registros (RecordSets) por tabla	**Excelente**; modela de forma natural relaciones complejas del ciclo de vida del huésped (Reserva → Habitación → Folio → Garantía).	Mapa de dominio, ciclo de vida del huésped (desde la reserva, garantías, hasta la inspección exprés y limpieza
Estados e invariantes	Difícil de proteger; deja la consistencia en manos de validaciones dispersas en scripts o formularios.	Parcial; protege restricciones a nivel de filas o tablas mediante restricciones de base de datos.	**Robusta protección de invariantes**; las entidades y agregados impiden estados ilegales (ej. prohibir sobreventa absoluta, asegurar que una habitación sucia no se asigne)	BR-16, BR-17, BR-19 (Overbooking cero, habitaciones fuera de servicio y bloqueo estricto hasta limpieza)
Criterio adicional	Bajo nivel de abstracción; alejado del lenguaje ubicuo del negocio hotelero.	Alineado a bases de datos tabulares, pero poco expresivo para reglas condicionales complejas de hotelería	Totalmente alineado al Lenguaje Ubicuo (Estándar, Suite, Folio, No-show, Garantía Incidental).	Glosario de Dominio y BRD (terminología estandarizada de procesos hoteleros)

Producto de la etapa: una decisión argumentada de una a dos páginas. No basta escribir que un patrón es más fácil, moderno o escalable. La explicación debe indicar qué problema concreto del caso ayuda a resolver y qué costo introduce.

Cisco Confidential

8 Etapa 3 Construir el mapa de dominio

Transformen el lenguaje y las reglas de la Unidad 2 en un mapa de responsabilidades. Identifiquen los elementos que el sistema debe reconocer, proteger o comunicar. No conviertan automáticamente cada tabla de datos en una entidad del dominio.

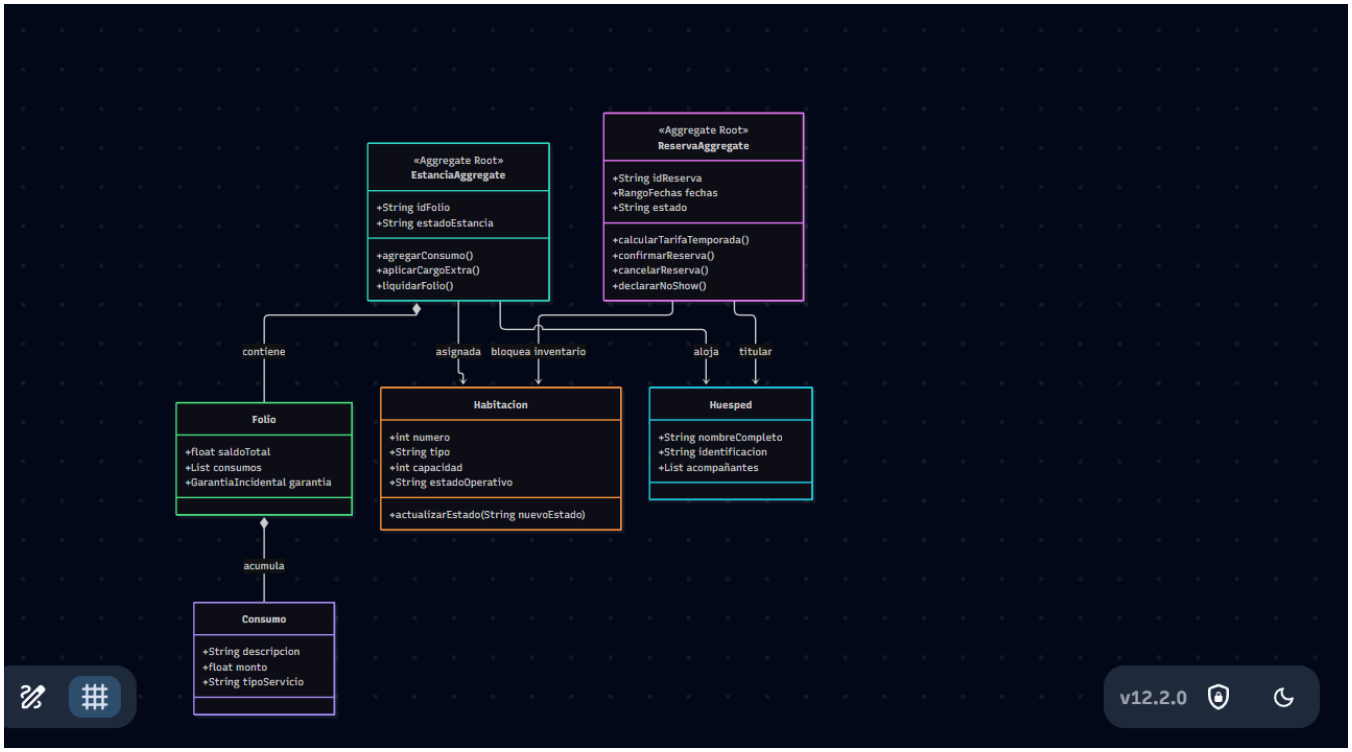
Elementos que deben identificar

Elemento	Pregunta que debe responder el equipo
Lenguaje ubicuo	Términos usados por usuarios, analistas y desarrolladores con el mismo significado.
Entidades	Conceptos con identidad y ciclo de vida que deben distinguirse aun cuando cambien sus datos.
Value Objects	Conceptos definidos por sus valores y reglas, sin identidad propia relevante.
Aggregates	Grupos que requieren un límite de consistencia y una raíz para controlar cambios.
Domain Services	Comportamientos del dominio que no pertenecen naturalmente a una sola entidad.
Domain Events	Hechos del negocio que ya ocurrieron y pueden interesar a otros componentes.

Producto de la etapa

- Diagrama del mapa de dominio con relaciones y límites claramente identificados.
- Glosario actualizado con los términos nuevos o corregidos.
- Ficha de cada Aggregate que indique su raíz, responsabilidades, invariantes y eventos posibles.
- Explicación de las decisiones dudosas o descartadas.

Cisco Confidential



Elementos del Mapa de Dominio

Lenguaje Ubicuo: Términos estandarizados de nuestro glosario y BRD: *Habitación (Estándar, Doble, Suite), Reserva, Garantía, Anticipo, No-show, Check-in, Early Check-in, Late Check-out, Folio, Garantía Incidental, Inspección Exprés, Limpia/Inspeccionada, Fuera de Servicio*.

Entidades (Tienen identidad propia y ciclo de vida):

Habitación: Mantiene un número único, tipo, capacidad y estado operativo actual (*Limpia/Inspeccionada*, *Sucia*, *Fuera de Servicio*).

Reserva: Registra los datos del huésped, fechas de estancia, tipo de habitación, estado (*Pendiente*, *Confirmada*, *Cancelada*, *No-Show*) y montos de anticipo.

Huésped: Contiene la información personal y documentos de identificación oficiales (INE/Pasaporte).

Folio: Cuenta contable asociada a la estancia que acumula los cargos por hospedaje y consumos adicionales.

Value Objects (Definidos por sus valores, sin identidad propia):

RangoFechas: Fecha de check-in y check-out con validación de plazos y temporadas.

MontoMonetario: Importes en MXN para tarifas, anticipos y penalizaciones.

GarantiaIncidental: Voucher abierto de tarjeta ($1,000 MXN) o efectivo ($500 MXN).

Cisco Confidential

DetalleConsumo: Descripción, categoría (restaurante, lavandería, estacionamiento), monto y firma digital o física.

Aggregates (Límites de consistencia y Raíces):

Agregado ReservaAggregate (Raíz: Reserva): Controla la consistencia entre fechas, disponibilidad de inventario, cálculo de tarifas por temporada y aplicación de garantías/cancelaciones sin permitir sobreventa (*overbooking cero*).

Agregado EstanciaAggregate (Raíz: Folio): Agrupa al huésped, la habitación asignada, el registro de consumos y la garantía incidental durante el ciclo de vida de la estancia, garantizando que no se cierre cuenta sin inspección exprés.

Domain Events (Hechos de negocio ocurridos):

ReservaConfirmada: Ocurre al registrar el anticipo del 50% o tarjeta en garantía.

ReservaCancelada: Emitido al cancelar con más o menos de 48 horas de anticipación.

NoShowDeclarado: Activado automáticamente a las 23:59 del día de llegada si no hay check-in.

CheckInCompletado: Registra identidad y garantía incidental, asignando habitación lista.

ConsumoAgregado: Registra un nuevo cargo directamente en el folio.

HabitacionFueraDeServicio: Marcada por Administración o Mantenimiento, descontando el inventario de inmediato.

Cisco Confidential

9 Etapa 4 Diseñar la Service Layer

Diseñen los Application Services que expondrán los casos de uso. Cada método debe representar una intención completa del usuario o de otro sistema. El servicio coordina la operación: recibe la solicitud, obtiene los objetos necesarios, invoca el comportamiento del dominio, solicita la persistencia y devuelve un resultado.

La Service Layer no debe convertirse en un lugar donde se acumulen todas las reglas. Si una condición describe lo que el negocio permite, calcula o protege, el equipo debe justificar dónde vive esa regla y por qué.

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
ReservaService	consultarDisponibilidad	Fechas (inicio, fin), cantidad de huéspedes, tipo de habitación	Repositorio de Habitaciones, Repositorio de Reservas	Lista de habitaciones compatibles y disponibles	Filtrado por capacidad y fechas para evitar duplicidades.
ReservaService	confirmarReserva	ID de reserva, datos de garantía (anticipo 50% o tarjeta de crédito)	Entidad Reserva, Pasarela de Pagos / Gestor de Garantías	Confirmación de reserva (Estado: Confirmado)	Validación del monto del anticipo o registro de tarjeta en garantía.
ReservaService	cancelarReserva	ID de reserva, fecha actual de cancelación	Entidad Reserva	Monto de reembolso o penalización calculado	Validación del plazo de 48 horas de anticipación para reembolso total (100%) o parcial (50%).
ReservaService	gestionarNoShow	Fecha y hora actual (ej. 23:59)	Repositorio de Reservas, Entidad Habitación	Listado de reservas declaradas No-Show y habitaciones liberadas	Declaración automática a las 23:59, liberación de habitación y cobro de la primera noche.
CheckInService	realizarCheckIn	ID de reserva, datos de identificación (INE/Pasaporte), acompañantes, garantía	Entidad Reserva, Entidad Estancia, Repositorio de Huéspedes	Folio de estancia creado y llave asignada	Registro de identidad y validación de la garantía incidental ($1,000 MXN o $500 MXN en efectivo).
CheckInService	gestionarEarlyCheckIn	ID de reserva, hora de llegada, estado de la habitación	Entidad Estancia, Entidad Habitación	Cargo adicional aplicado al folio (30% si llega antes 12:00)	Aplicación del recargo del 30% de la tarifa diaria si hay disponibilidad anticipada.
CheckOutService	gestionarLateCheckOut	ID de estancia, hora solicitada de salida	Entidad Estancia	Cargo calculado en el folio	Regla de 30% de cargo hasta las 15:00 hrs, o noche completa posterior a esa hora.
CheckOutService	procesarSalidaAnticipada	ID de estancia, fecha de aviso de salida	Entidad Estancia, Folio	Cuenta final y penalización aplicada	Aplicación de política según aviso previo de 24 horas (noches hospedadas + media noche) o mismo día (noche completa).

Cisco Confidential

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
HabitacionService	cambiarHabitacionPorAveria	ID de estancia, motivo de avería, nueva habitación asignada	Entidad Estancia, Entidad Habitación	Reubicación exitosa	Reubicación en categoría igual o superior sin costo adicional ante fallas no subsanables.
HabitacionService	procesarUpgradeVoluntario	ID de estancia, habitación destino (Suite)	Entidad Estancia, Entidad Habitación	Diferencia prorrateada cargada al folio	Cálculo y cobro proporcional de la diferencia de tarifa por las noches restantes.
ConsumoService	cargarConsumoAlFolio	ID de folio / habitación, concepto (restaurante, lavandería), monto, firma	Entidad Folio, Repositorio de Consumos	Cargo registrado en cuenta consolidada	Registro inmediato de consumos en el folio activo.
LimpiezaService	solicitarInspeccionCheckout	ID de habitación / folio	Servicio de Notificación a Limpieza, Entidad Habitación	Solicitud de inspección exprés enviada	Activación de la revisión exprés de 5 minutos (frigobar y daños) previa al cierre de cuenta.
LimpiezaService	actualizarHabitacionLimpia	ID de habitación, nuevo estado ('Limpia/Inspeccionada')	Entidad Habitación	Estado de habitación actualizado en tiempo real	Habilitación de la habitación para asignación de recepción únicamente tras la inspección.
MantenimientoService	bloquearFueraDeServicio	ID de habitación, motivo, fecha estimada de reactivación	Entidad Habitación, Inventario	Habitación bloqueada y retirada del inventario	Descuento inmediato del inventario operativo y registro de la causa.
AdministracionService	autorizarOperacionSensible	Tipo de operación (descuento >10%, condonación, anulación), PIN de administración	Entidad de Autorización, Roles de Usuario	Operación autorizada / denegada	Validación obligatoria de PIN administrativo para operaciones con restricciones financieras.

Cisco Confidential

Revision obligatoria

- ¿El nombre del método expresa una acción del negocio?
  Sí, cada método utiliza verbos y conceptos del dominio hotelero (consultarDisponibilidad, realizarCheckIn, procesarUpgradeVoluntario).
- ¿El método coordina o contiene decisiones que deberían estar en el dominio?
  La capa de aplicación únicamente orquesta (recibe la entrada, busca los repositorios, invoca el método correspondiente de la entidad y persiste), dejando los cálculos de tarifas por temporada, penalizaciones y reglas de negocio encapsuladas en el dominio.
- ¿El servicio depende de interfaces o de detalles concretos de base de datos?
  Depende de abstracciones (repositorios e interfaces de dominio), manteniendo el diseño desacoplado.
- ¿La operación tiene un inicio y un final transaccional comprensible?
  Sí, cada caso de uso representa una transacción atómica completa del ciclo del huésped.
- ¿Los errores del dominio pueden explicarse con términos del negocio?
  Sí, manejan excepciones orientadas al dominio como habitaciones no disponibles, no-show, falta de garantía o PIN inválido.

Cisco Confidential

10 Etapa 5 Separar persistencia y transacciones

Definan interfaces de Repository en términos del dominio y describan, por separado, cómo podrían implementarse en infraestructura. Los métodos deben expresar lo que el dominio necesita recuperar o guardar, sin exponer consultas SQL como parte del modelo.

También indiquen qué operaciones deben completarse juntas. La propuesta de Unit of Work debe explicar dónde inicia la transacción, qué cambios incluye y qué ocurriría ante un error.

Repository	Necesidad del dominio	Metodos propuestos	Objeto que recupera o guarda
HabitacionRepository	Consultar habitaciones libres, verificar estados actuales y gestionar inventario físico o fuera de servicio.	buscarPorId(id), buscarDisponibles(fechas, tipo, capacidad), guardar(habitacion), actualizarEstado(id, estado)	Habitacion
ReservaRepository	Registrar apartados, validar duplicidades, procesar anticipos, cancelaciones y control de no-shows.	buscarPorId(id), buscarPorFechaLlegadaYEstado(fecha, estado), guardar(reserva), actualizar(reserva)	Reserva
EstanciaRepository	Controlar el ingreso (check-in), asignación de llaves, garantía incidental y seguimiento de la estancia.	buscarPorId(id), buscarActivaPorHabitacion(habitacionId), guardar(estancia), actualizar(estancia)	Estancia
FolioRepository	Consolidar consumos adicionales (restaurante, lavandería) y cargos calculados de la cuenta del huésped.	buscarPorId(id), buscarPorEstancia(estancialId), guardar(folio), agregarCargo(folioId, cargo)	Folio
AutorizacionRepository	Validar credenciales y permisos de administración para operaciones sensibles (descuentos >10%, etc.).	validarPinAdministrativo(pin)	CredencialAdministrativa / Boolean

Cisco Confidential

PROPUESTA DE UNIT OF WORK Y MANEJO DE TRANSACCIONES

1. Dónde inicia la transacción:

La transacción inicia explícitamente en la capa de aplicación (Service Layer) justo antes de invocar los métodos de modificación de los repositorios involucrados (por ejemplo, al ejecutar realizarCheckIn o confirmarReserva).

2. Qué cambios incluye: Agrupa de forma atómica múltiples actualizaciones. Por ejemplo, en el caso de uso realizarCheckIn (CheckInService), se incluye en la misma transacción:
- La actualización del estado de la Reserva (a registrada/atendida). - La creación del nuevo registro de Estancia y Folio asociado.
- El registro de los datos de identificación del titular, acompañantes y la garantía incidental. Todas estas modificaciones se consolidan y confirman juntas.

3. Qué ocurriría ante un error:

Si ocurre una excepción durante la ejecución (por ejemplo, fallo en la pasarela de pagos, garantía incidental insuficiente o indisponibilidad de la habitación), el Unit of Work ejecuta un rollback automático de todas las operaciones realizadas. Esto garantiza la integridad de los datos, evitando estados inconsistentes como tener una reserva confirmada sin su anticipo cobrado o una estancia creada sin habitación asignada.

Cisco Confidential

11 Etapa 6 Separar Commands y Queries

Apliquen CQRS básico para hacer explícita la diferencia entre cambiar el estado y consultar información. No es obligatorio construir bases de datos separadas. El objetivo es que cada operación tenga una responsabilidad clara y que las consultas puedan devolver un modelo apropiado para lectura.

Trabajo solicitado

- Seleccionen al menos dos operaciones que cambien el estado y descríbanlas como Commands.
- Seleccionen al menos dos necesidades de información y descríbanlas como Queries.
- Definan el Handler de cada operación y sus dependencias.
- Diseñen el DTO o View Model de lectura sin reutilizarlo automáticamente como modelo de escritura.
- Justifiquen si alguna parte del caso no necesita esta separación.

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificacion
Command	ConfirmarReservaCommand	reservaId, montoAnticipo, metodoPago, datosGarantia	ConfirmarReservaHandler	ReservaConfirmadaResult	Modifica el estado del dominio (cambia Reserva de 'Pendiente' a 'Confirmada' y aplica cobro). Requiere validación de reglas de negocio y transaccionalidad.
Command	RealizarCheckInCommand	reservaId, documentoIdentidad, acompañantes, garantiaIncidental	RealizarCheckInHandler	CheckInCompletadoResult	RealizarCheckInHandler | CheckInCompletadoResult | Modifica el estado del dominio (crea la Estancia activa, asigna llave y crea Folio). Debe ejecutar validaciones y persistencia atómica.

Cisco Confidential

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificacion
Query	ConsultarDisponibilidadQuery	fechaInicio, fechaFin, tipoHabitacion, cantidadHuespedes	ConsultarDisponibilidadHandler	List<HabitacionDisponibleDTO>	ConsultarDisponibilidadHandler| List<HabitacionDisponibleDTO>| Operación de solo lectura. No modifica el estado de las entidades. Optimiza la consulta devolviendo únicamente datos relevantes para la selección.
Query	ObtenerOcupacionDiariaQuery	fechaConsulta	ObtenerOcupacionDiariaHandler	OcupacionDiariaViewModel	Operación de solo lectura que consolida métricas (entradas, salidas, fuera de servicio). Evita cargar entidades complejas de dominio para un reporte.

Cisco Confidential

1. Definición de Handlers y Sus Dependencias:

- ConfirmarReservaHandler:

* Dependencias: ReservaRepository, PasarelaPagosService, UnitOfWork.

* Función: Carga el agregado Reserva, invoca su método de confirmación con la garantía, delega el cobro del anticipo a la pasarela y guarda los cambios transaccionalmente.

- RealizarCheckInHandler:

* Dependencias: ReservaRepository, EstanciaRepository, FolioRepository, UnitOfWork.

* Función: Verifica la reserva, crea la entidad Estancia y su Folio asociado, y registra la garantía incidental.

- ConsultarDisponibilidadHandler:

* Dependencias: HabitacionReadRepository / IDbConnection (Consulta directa optimizada de lectura).

* Función: Ejecuta una consulta ligera a la base de datos para recuperar habitaciones libres en el rango especificado sin pasar por lógica de dominio compleja.

- ObtenerOcupacionDiariaHandler:

* Dependencias: ReporteReadRepository.

* Función: Realiza una proyección/agregación directa de datos para construir el View Model del reporte operativo diario.

2. Diseño de DTOs / View Models de Lectura:

- HabitacionDisponibleDTO:

* Propiedades: habitacionId, numero, tipoHabitacion, capacidadMax, tarifaPorNoche, amenidades.

* Nota: Es una proyección plana sin comportamientos ni métodos de negocio.

- OcupacionDiariaViewModel:

* Propiedades: fecha, totalHabitaciones, totalOcupadas, totalEntradasProgramadas, totalSalidasProgramadas, listaFueraDeServicio.

* Nota: Diseñado exclusivamente para visualizar métricas ejecutivas/administrativas sin instanciar modelos de dominio pesados.

3. Justificación de Partes que NO Necesitan Separación (CQRS Completo / Bases de datos separadas):

- Justificación: En el sistema del Hotel Costa Azul no se justifica implementar bases de datos físicamente separadas (una de lectura y otra de escritura) ni sincronización asíncrona por eventos (Event Sourcing), ya que la escala de concurrencia y volumen de datos de un hotel de esta categoría no sobrepasa la capacidad de una base de datos relacional estándar.

- La separación de Commands y Queries a nivel de código (métodos/handlers diferenciados y DTOs de lectura) es suficiente para mantener un código limpio, desacoplado y optimizado sin agregar la complejidad operacional de sincronizar dos bases de datos.

Cisco Confidential

12 Etapa 7 Revisar la propuesta con SOLID

Se revisó la propuesta completa (mapa de dominio, Service Layer, Repository/Unit of Work y CQRS). Se registran siete hallazgos: tres confirman buenas decisiones y cuatro obligaron a corregir el diseño.

#	Principio	Evidencia	Impacto	Cambio realizado / decisión
H-01	S	ReservaAggregate se definió como responsable de la consistencia entre fechas, inventario, tarifas, garantías y cancelaciones (BR-19, BR-05, BR-06). Mezcla el ciclo de vida de una reserva con el control de inventario global.	Dos razones de cambio. El overbooking cero (BR-19) no se puede proteger desde una sola Reserva y se arriesga a sobreventa.	Corrección (C-02): Reserva solo protege su estado, garantía y fechas. ServicioDisponibilidad (Domain Service) bloquea la habitación de forma atómica al reservar.
H-02	S	Los métodos de ReservaService, CheckInService y CheckOutService solo reciben datos, cargan agregados, invocan el dominio y guardan. Los cálculos (35%, 30%, 48 h, 24 h) no aparecen en la capa de aplicación.	Confirma la separación coordinación / reglas. Un cambio de política no obliga a tocar servicios.	Se confirma. Excepción justificada: la validación del PIN (BR-18) es una autorización de aplicación y se hace en el servicio mediante IAutorizacionRepository.
H-03	O	Las reglas BR-04 (35%), BR-06 (48 h / 50%), BR-09/BR-10 (30%) y BR-13 (24 h) son umbrales de negocio que pueden cambiar (se usaron como evidencia de cambios esperados en la Etapa 2). Resolverlas con if/else obliga a editar cadenas de condiciones por cada nueva política.	Alto riesgo de modificar código estable cada vez que cambia una tarifa, un plazo o un porcentaje.	Se definen IPoliticaCancelacion, IPoliticaCargoHorario e IPoliticaSalidaAnticipada. Cada regla es una implementación; los porcentajes y umbrales viven en Value Objects (Temporada, MontoMonetario). Ver ADR-003.
H-04	L	BR-05 permite dos garantías de reserva (anticipo del 50% o tarjeta). BR-07 exige cobrar la primera noche «a la tarjeta en garantía». Un subtipo «Anticipo» no puede cumplir esa operación.	Si ambas garantías comparten una interfaz, el subtipo Anticipo tendría que lanzar error o ignorar la operación y se violaría el contrato.	Se define el contrato GarantiaReserva.cobrarPrimeraNoche() que devuelve un resultado explícito (retener anticipo / cargar a tarjeta). Qué se hace con anticipo sin tarjeta queda PENDIENTE DE VALIDACIÓN (H-U2-02); no se asume una regla.
H-05	I	IHabitacionRepository acumulaba consultas de disponibilidad, cambio de estado, inventario y bloqueo. LimpiezaService solo necesita cambiar el estado y los reportes solo leen.	Los consumidores dependen de métodos que no usan; cualquier cambio de consultas afecta a los servicios de escritura.	IHabitacionRepository queda con buscarPorId, buscarAsignable y guardar (escritura). Las consultas pasan a IDisponibilidadReadModel (lado de lectura, CQRS). La autorización se mantiene en IAutorizacionRepository.
H-06	D	ConsultarDisponibilidadHandler dependía de IDbConnection; el handler de confirmación dependía de una pasarela de pagos concreta; el no-show a las 23:59 dependía de la hora del sistema. El BRD (4.2) deja fuera proveedores de pago.	Aplicación atada a SQL y a un proveedor no definido; pruebas de 23:59 y de umbral de 48 h no deterministas.	Se introducen puertos: IDisponibilidadReadModel (lectura), IGestorGarantias (cobros y reembolsos) e IReloj (hora). La infraestructura los implementa (DisponibilidadReadRepository, GestorGarantiasAdapter, RelojSistema).

Cisco Confidential

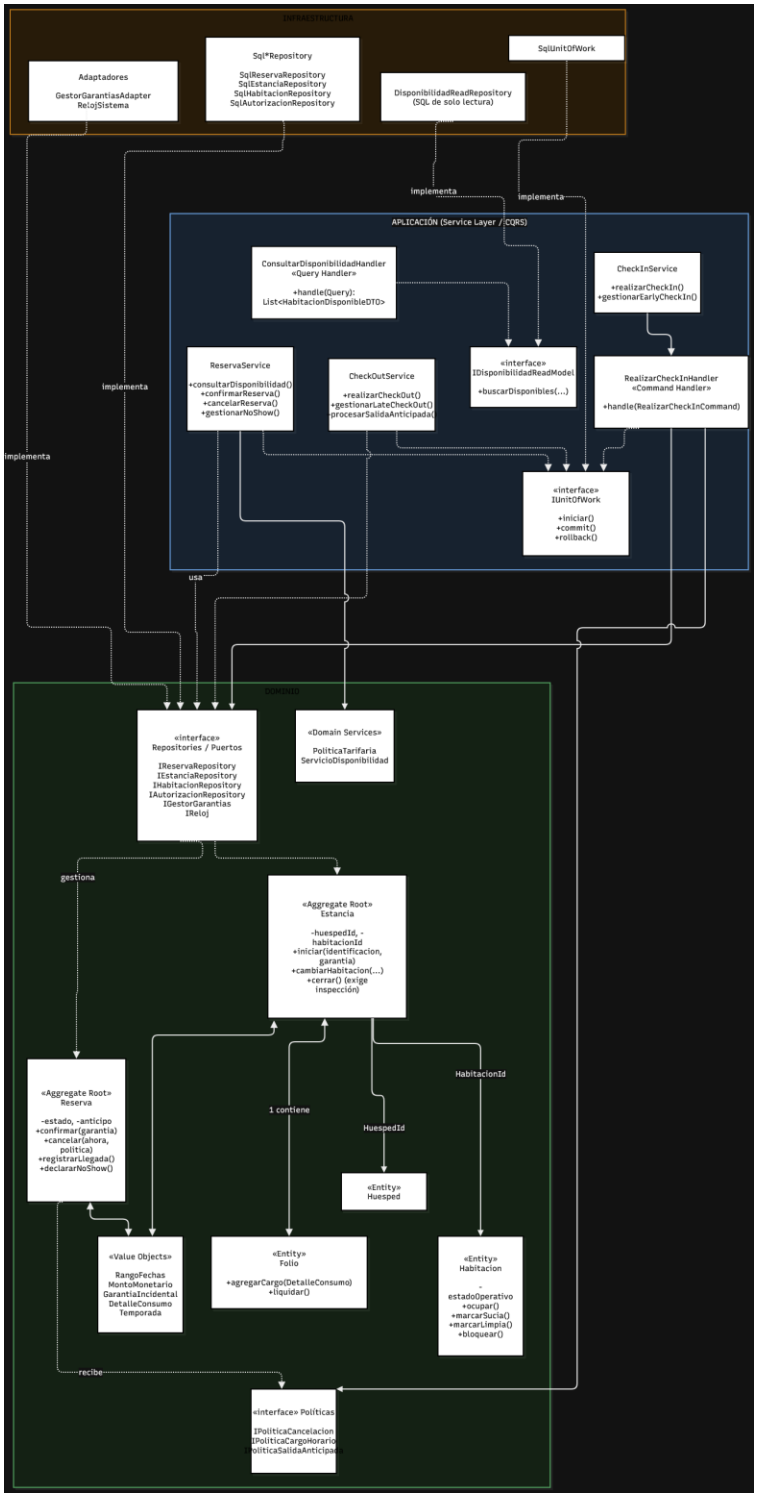
#	Principio	Evidencia	Impacto	Cambio realizado / decisión
H-07	D	La Unit of Work revierte la base de datos, pero no puede deshacer un reembolso ya enviado a un sistema externo.	Podría registrarse un reembolso sin que la cancelación quede persistida, o al revés.	El reembolso se ejecuta después del commit a partir del evento ReservaCancelada (ver secuencia 2). Si falla, queda pendiente de reintento y no se pierde la cancelación.

Cisco Confidential

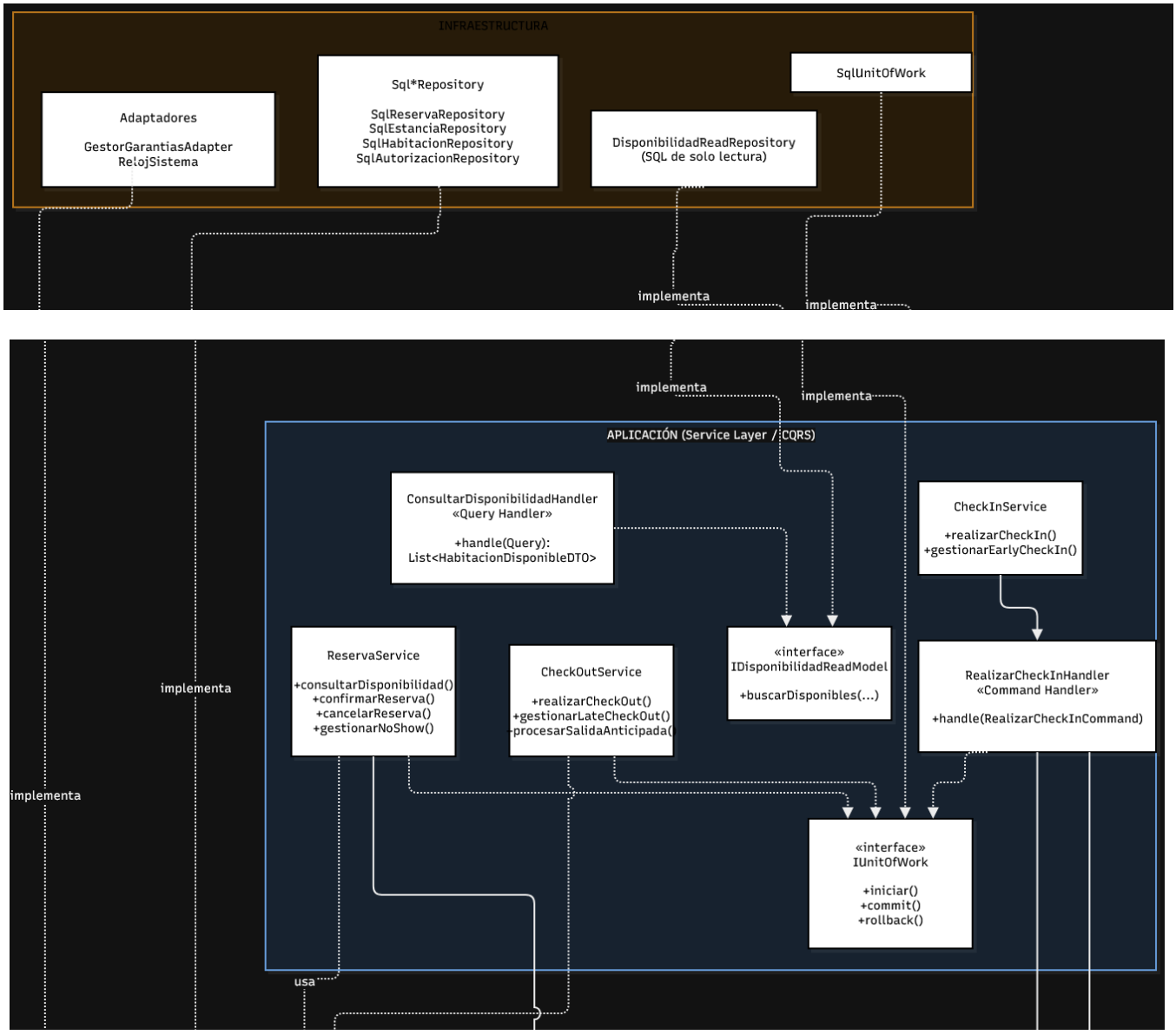
13 Etapa 8 Representar y documentar la arquitectura

Diagrama de clases (dominio, aplicación e infraestructura)

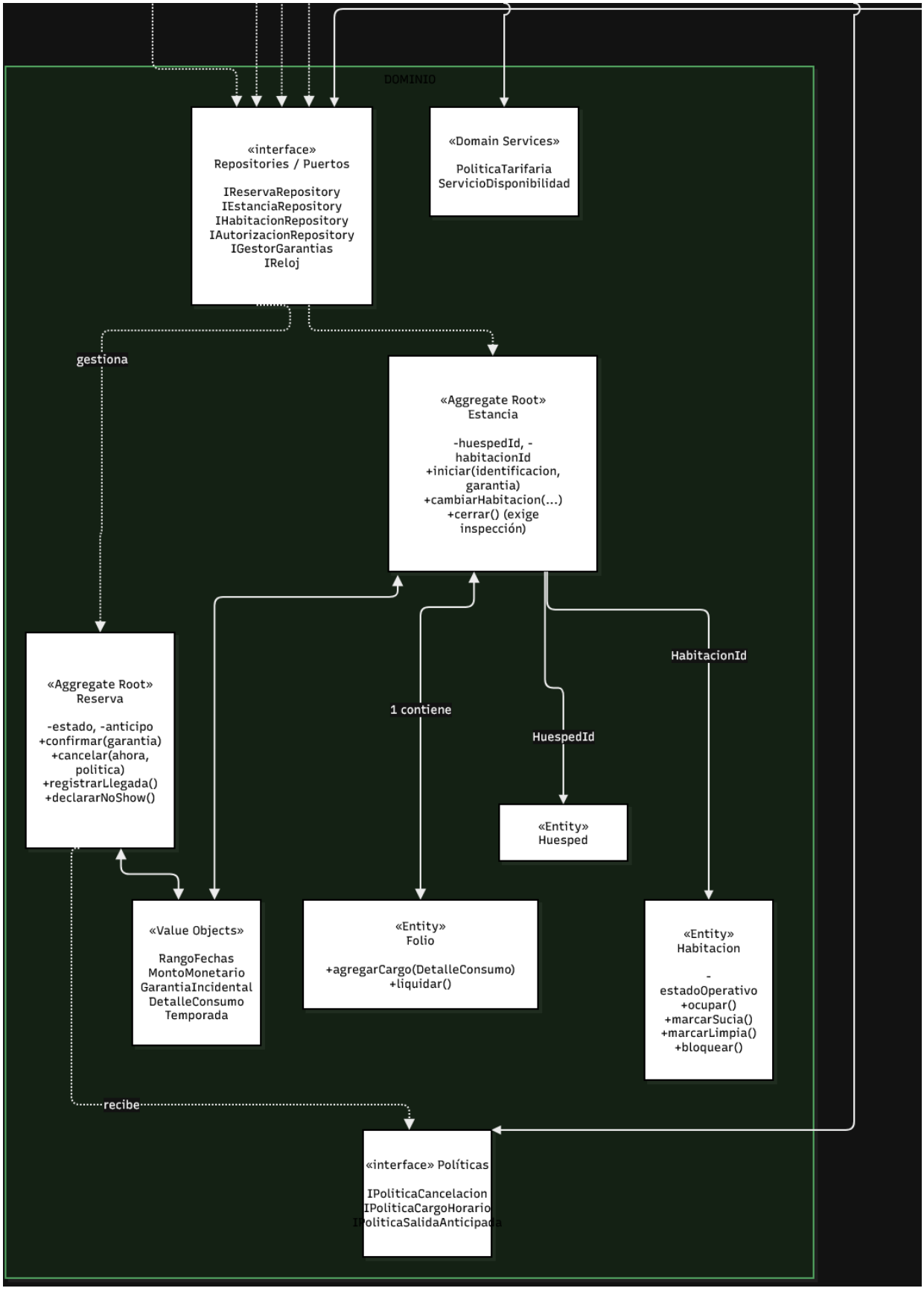
Capas diferenciadas por color: azul = Aplicación, verde = Dominio, naranja = Infraestructura. Las flechas punteadas con triángulo vacío indican implementación; las punteadas con flecha, dependencia. La infraestructura depende del dominio, no al revés.



Cisco Confidential



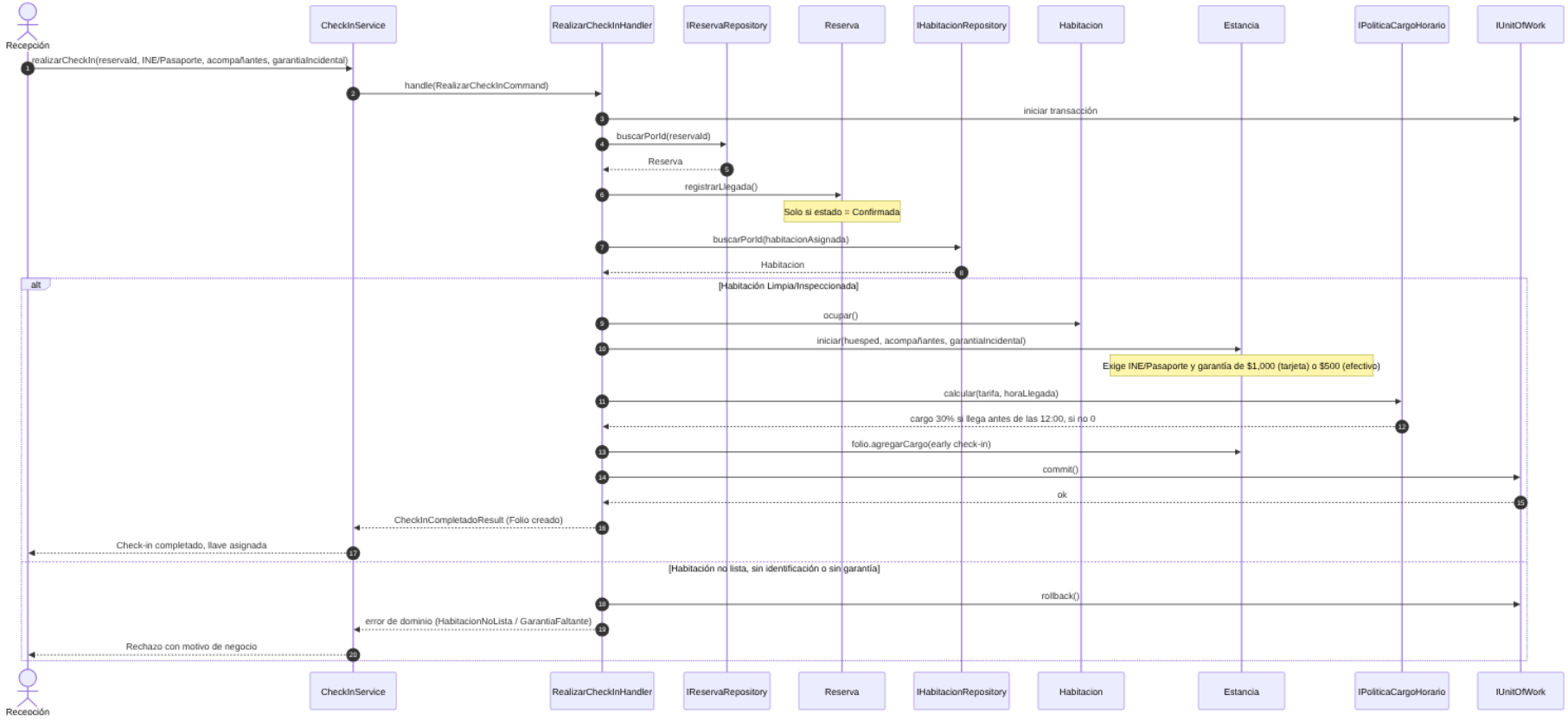
Cisco Confidential



Cisco Confidential

Diagrama de secuencia 1 – Flujo principal: CU-03 Realizar check-in

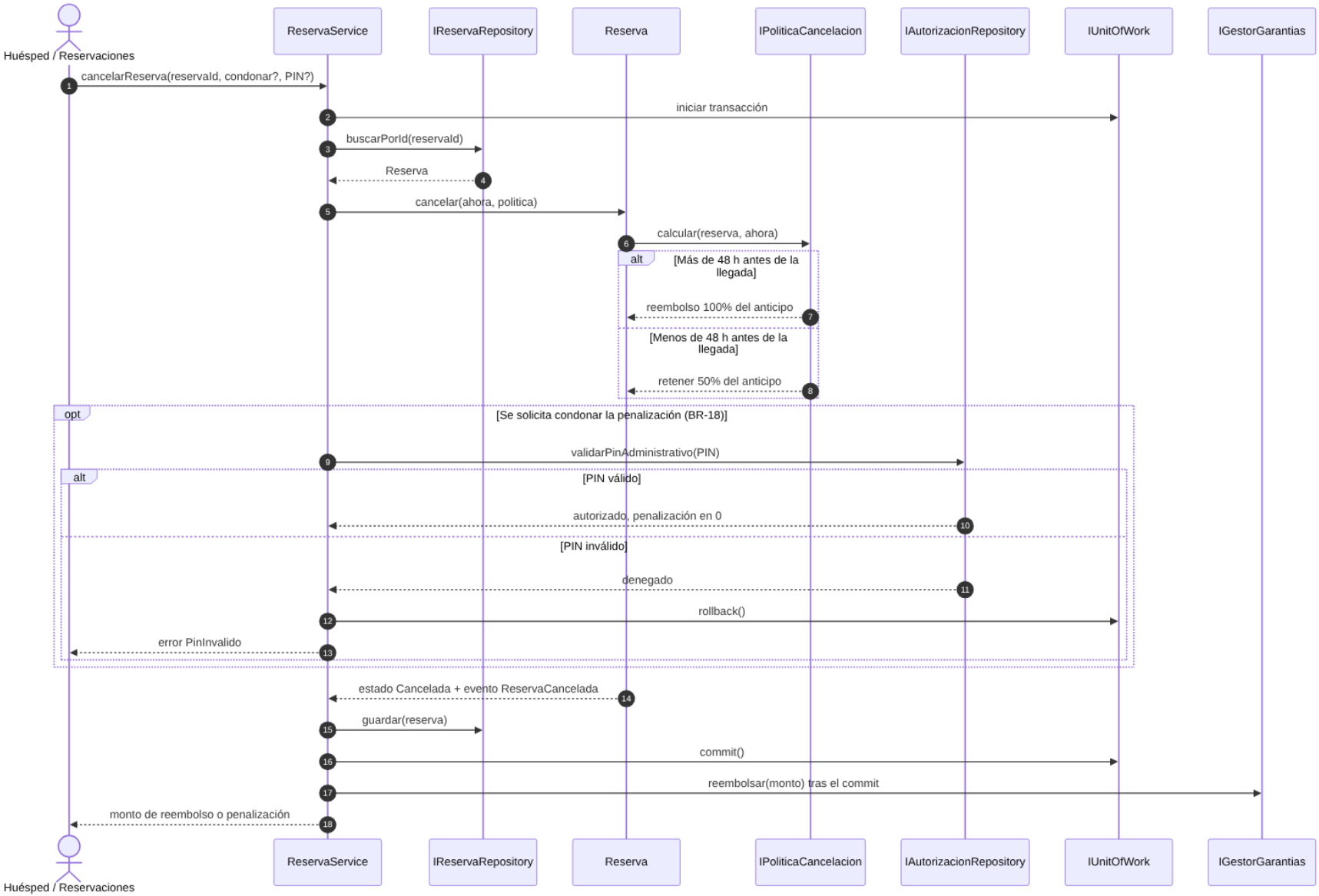
Incluye la alternativa de rechazo (habitación no lista, sin identificación o sin garantía) con rollback. Reglas: BR-08, BR-09, BR-11, NB-07.



Cisco Confidential

Diagrama de secuencia 2 – Flujo con regla y excepción: CU-02 Cancelar reserva

Incluye la regla de 48 horas (BR-06), la condonación con PIN de Administración (BR-18) y el reembolso posterior al commit (H-07).



Cisco Confidential

Architecture Decision Records (ADR)

ADR-001 – Domain Model como patrón principal de lógica de negocio

Campo	Contenido
Estado	Aceptada.
Contexto	Las reglas dependen de tiempo, estado y umbrales: tarifa por temporada +35% (BR-04), cancelación a 48 h (BR-06), no-show a las 23:59 (BR-07), early check-in y late check-out con 30% (BR-09, BR-10), salida anticipada a 24 h (BR-13). Además hay invariantes: overbooking cero (BR-19) y habitación vendible solo si está limpia e inspeccionada (NB-07, BR-16).
Alternativas	Transaction Script: simple, pero duplicaría umbrales en CU-02, CU-03 y CU-04 y dejaría las invariantes en validaciones dispersas. Table Module: orientado a tablas, acopla la lógica al esquema y protege poco los estados. Domain Model: encapsula reglas y estados en objetos con el lenguaje del glosario.
Decisión	Domain Model para reservas, estancias, folios y habitaciones. Los reportes (corte de caja, folios abiertos, ocupación) se resuelven en el lado de lectura (consultas directas con DTO), sin pasar por el dominio.
Consecuencias	Beneficios: invariantes protegidas, lenguaje ubicuo y cambios de política localizados. Costos: más diseño inicial, necesidad de repositorios y mapeo, curva de aprendizaje del equipo. Riesgo: sobrediseñar partes simples; se mitiga manteniendo los reportes fuera del dominio.
Trazabilidad	NB-01, NB-02, NB-03, NB-07; BR-04, BR-06, BR-07, BR-09, BR-10, BR-13, BR-16, BR-19; US-02 a US-08; CU-01 a CU-04.

ADR-002 – Límites de agregados: Reserva, Estancia (con Folio) y Habitación; disponibilidad como Domain Service

Campo	Contenido
Estado	Aceptada (corrige la Etapa 3, ver C-01 y C-02).
Contexto	La reserva existe antes de la estancia; la habitación tiene ciclo propio (limpieza, fuera de servicio); el overbooking cero (BR-19) cruza varias reservas.
Alternativas	(a) Un solo agregado Reserva–Estancia–Folio: límite enorme y bloqueos largos. (b) Folio como raíz: la Estancia quedaría sin dueño y los servicios ya la usan. (c) Habitación dentro de Estancia: impediría bloquearla o limpiarla de forma independiente.
Decisión	Tres agregados independientes: Reserva, Estancia (raíz) con su Folio, y Habitación. Se referencian por identidad. ServicioDisponibilidad coordina el bloqueo atómico del inventario.
Consecuencias	El check-in modifica Reserva, Estancia y Habitación en una sola transacción de caso de uso (Unit of Work). Es un compromiso aceptado y documentado: el caso de uso, no el agregado, define la atomicidad. Si el volumen crece, se pasaría a eventos de dominio.
Trazabilidad	NB-01, NB-07, NB-08; BR-16, BR-17, BR-19; US-06, US-14, US-15; CU-01, CU-03, CU-04.

Cisco Confidential

ADR-003 – Políticas de cargo y penalización como estrategias de dominio

Campo	Contenido
Estado	Aceptada. Pendiente de validación: BR-07 con anticipo sin tarjeta (H-U2-02).
Contexto	Los porcentajes y plazos (35%, 30%, 50%, 48 h, 24 h) son parámetros de negocio con riesgo de cambio (BR-02, BR-04, BR-10 en la Etapa 2).
Alternativas	(a) if/else dentro de los servicios: viola OCP y duplica reglas. (b) Parámetros solo en configuración, sin comportamiento: no cubre reglas con forma distinta (por ejemplo, «noche completa» frente a «30%»). (c) Interfaces de política con implementaciones y parámetros en Value Objects.
Decisión	Se usan IPoliticaCancelacion, IPoliticaCargoHorario e IPoliticaSalidaAnticipada. Cada regla es una clase; los valores viven en Value Objects (Temporada, MontoMonetario).
Consecuencias	Más clases pequeñas, pero se puede cambiar o agregar una política sin editar los servicios ni los agregados. Las pruebas unitarias son directas por regla. Costo: hay que mantener el registro de qué política aplica en cada caso.
Trazabilidad	NB-02, NB-03; BR-04, BR-06, BR-09, BR-10, BR-13; US-02, US-04, US-07, US-08, US-11.

ADR-004 – CQRS básico sin base de datos separada y Unit of Work en la Service Layer

Campo	Contenido
Estado	Aceptada.
Contexto	El hotel tiene 120 habitaciones. La disponibilidad debe ser consistente al segundo (BR-19) y los reportes de medianoche (BR-20) solo leen.
Alternativas	(a) CQRS completo con bases separadas y eventos: complejidad operativa no justificada y riesgo de lecturas desfasadas. (b) Sin separación: los reportes cargarían agregados completos. (c) Separación en código: Commands/Queries con Handlers distintos y DTO de lectura, misma base de datos.
Decisión	Opción (c). Cada Command Handler es una transacción (Unit of Work) que inicia en la capa de aplicación; las Queries usan puertos de lectura y devuelven DTO/ViewModels. Los efectos externos (reembolsos) se disparan después del commit.
Consecuencias	Lecturas siempre consistentes y sin infraestructura extra. Hay que mantener dos modelos (escritura y DTO de lectura). Si el negocio crece, el lado de lectura puede moverse a otra base sin cambiar el dominio.
Trazabilidad	NB-01, NB-10; BR-19, BR-20; US-01; CU-01.

Cisco Confidential

ADR-005 – Puertos para pasarela de pagos, reloj y autorizaciones

Campo	Contenido
Estado	Aceptada.
Contexto	El BRD (4.2) deja fuera proveedores de pago e integraciones. Varias reglas dependen del tiempo: 23:59, 12:00, 15:00, 48 h, 24 h.
Alternativas	(a) Llamar directamente a una pasarela y a la hora del sistema desde los servicios. (b) Interfaces (puertos) en el dominio/aplicación y adaptadores en infraestructura.
Decisión	IGestorGarantias, IReloj e IAutorizacionRepository como puertos; GestorGarantiasAdapter y RelojSistema como adaptadores iniciales (simulados).
Consecuencias	Pruebas deterministas de no-show y plazos, y proveedor de pagos reemplazable sin tocar el dominio. Costo: se necesita un adaptador simulado en el código base.
Trazabilidad	NB-03, NB-09; BR-06, BR-07, BR-18; US-04, US-05, US-16.

Cisco Confidential

14 Vibe Coding con IA

Usarán IA para convertir descripciones del negocio en alternativas de arquitectura, generar estructura inicial, revisar código y producir borradores de diagramas. El equipo sigue siendo responsable de la arquitectura y debe verificar cada resultado contra los documentos de la Unidad 2.

Ciclo de trabajo esperado

Paso	Accion	Evidencia
1	Preparar contexto	Seleccionar fragmentos relevantes del BRD, historias, reglas, BPMN y glosario.
2	Solicitar propuesta	Pedir una alternativa concreta y exigir que la IA declare supuestos.
3	Comparar	Contrastar la respuesta con el caso y con los conceptos de la unidad.
4	Corregir	Modificar nombres, responsabilidades, límites y dependencias que no correspondan.
5	Validar	Revisar trazabilidad, consistencia entre diagramas y comportamiento esperado.
6	Registrar	Guardar prompt, respuesta, decisión humana y versión resultante.

Cisco Confidential

Prompts de apoyo

Los siguientes formatos son puntos de partida. Completen los corchetes con información de su empresa y eviten enviar datos que no formen parte del caso académico.

- Patrones: Con base en [reglas, cambios y casos de uso], compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.
- Dominio: Revisa este glosario y estas reglas [contenido]. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Para cada candidato, indica la evidencia utilizada y las dudas que requieren validación humana.
- Service Layer: A partir de estos casos de uso [contenido], propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa.
- SOLID: Analiza este diseño [contenido] y señala posibles violaciones SOLID. No reescribas todo; identifica evidencia, riesgo y alternativas para que el equipo decida.
- Diagramas: Convierte esta arquitectura validada [contenido] en un borrador PlantUML o Mermaid. No agregues clases, reglas o relaciones que no aparezcan en la descripción.

Cisco Confidential

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
1	**ETAPA 1:** Preparar el caso y seleccionar los casos de uso clave del proceso principal a partir del BRD y las User Stories.	A partir del BRD y las User Stories del Hotel Costa Azul, selecciona 4 casos de uso que cubran flujos normales y excepciones (reservas, garantías, check-in, check-out y limpieza).	Propuesta de 4 casos de uso (CU-01 a CU-04) vinculados directamente con las reglas de negocio (BR-01 a BR-16)	Se asumió inicialmente agrupar check-in y early check-in en un solo caso, lo cual diluía las reglas de horario específicas.	Se separaron los flujos para mantener trazabilidad estricta con las reglas BR-08, BR-09 y BR-10 de la Unidad 2
2	**Etapa 2:** Comparar los patrones de lógica de negocio (*Transaction Script*, *Table Module* y *Domain Model*) con base en las evidencias del caso	Con base en las reglas de tarifas por temporada, overbooking cero e invariantes del Hotel Costa Azul, compara Transaction Script, Table Module y Domain Model	Tabla comparativa detallando complejidad de cambios, manejo de estados y protección de invariantes para cada patrón.	La IA sugirió inicialmente *Table Module* argumentando simplicidad tabular para las 120 habitaciones	Se corrigió a *Domain Model* porque el hotel maneja reglas complejas de negocio interconectadas (penalizaciones por hora, no-show a las 23:59 y estados de limpieza) que requieren encapsulamiento en objetos de dominio
3	**Etapa 3:** Construir el mapa de dominio, identificando lenguaje ubicuo, entidades, value objects, aggregates y domain events	*Prompt interno:* Revisa el glosario y el BRD del Hotel Costa Azul y propón candidatos a entidades, value objects, aggregates y domain events en formato Mermaid.	Estructura de clases en Mermaid y listado de elementos con sus respectivos límites de consistencia y raíces de agregados	La IA colocó los consumos directamente dentro del agregado de reservas en lugar de asociarlos al folio de la estancia	Se ajustó la arquitectura para separar el ReservaAggregate del EstanciaAggregate (centrado en el Folio y consumos), alineándolo con el ciclo de vida real del huésped
4	Etapa 4 Diseñar la Service Layer	"es un diseño puramente conceptual, ahora te estoy pasando un pdf con los user stories y otro con un glosario de dominio" (Adjuntando PDFs)	Tabla completa con 15 métodos de servicio, entradas, colaboradores, salidas y reglas delegadas para el ciclo de vida del hotel.	El asistente generó inicialmente un archivo de Google Docs con formato complejo que dificultaba la extracción directa.	El equipo decidió solicitar la información en formato de texto plano (*plain text*) para facilitar la copia y estructuración de la bitácora y entrega.

Cisco Confidential

5	Etapa 5 Separar Persistencia y Transacciones	"Ahora necesito la interfaces de repository, tu me dices si necesitas mas información o con los pdf del user story y el glosario es posible realizarlo"	Definición de 5 interfaces de repositorio en términos de dominio y la propuesta del patrón Unit of Work con manejo de rollback.	Las tablas automáticas del chat presentaban saltos de formato difíciles de alinear directamente en la plantilla de entrega.	El equipo solicitó nuevamente el contenido en formato de texto plano (*plain text*) para unificar el estilo de la documentación.
6	Etapa 6 Separar Commands y Queries (CQRS)	"Utiliza CQRS para realizar las siguientes instrucciones"	Diseño conceptual de Commands, Queries, Handlers, DTOs de lectura y la justificación de no requerir bases de datos separadas.	Suposición inicial de que se requería desarrollo de código funcional, cuando el entregable es puramente conceptual y de arquitectura.	El equipo decidió mantener un enfoque conceptual y de diseño de arquitectura, utilizando texto plano (*plain text*) para copiar los apartados en la bitácora.
7	Revisar los insumos de la U2 (BRD, glosario, historias) y el avance de la actividad U3 para detectar huecos.	«checa estos documentos» (con el .docx de la actividad, BRD, glosario e historias de usuario adjuntos).	Resumen del estado por etapa y 9 inconsistencias: Estancia usada en servicios pero ausente del mapa de dominio; Reserva controlando el inventario; falta de realizarCheckOut; falta de consultas de reportes (BR-20); eventos de dominio incompletos.	No se tenía el BPMN ni la matriz de trazabilidad de la U2, así que no se revisó contra ellos. Supuesto: los PDF de la U2 son la versión vigente.	Propuesta: aceptar las correcciones C-01 a C-05 y registrar H-U2-01 (falta de historias para check-out y reportes). Confirmar contra el BPMN.
8	Elaborar la Etapa 7 (SOLID) y la Etapa 9 (validación): hallazgos, recorrido de casos de uso y tabla de validación.	«puedes realizar el 7 a la 9».	7 hallazgos SOLID con evidencia, impacto y cambio (por ejemplo, políticas como estrategias, puertos IReloj / IGestorGarantias / IDisponibilidadReadModel, reembolso tras el commit). Recorrido de CU-01 a CU-04 con el componente que valida cada regla.	Supuesto señalado, no resuelto: BR-07 cobra el no-show a la «tarjeta en garantía», pero BR-05 permite confirmar solo con anticipo. Se marcó como pendiente de validación en lugar de inventar la regla (H-04, H-U2-02).	Propuesta: aceptar los hallazgos H-01 a H-07 tras revisarlos. Preguntar al profesor o stakeholder qué pasa con el no-show cuando solo hay anticipo.

Cisco Confidential

9	Elaborar la Etapa 8: diagrama de clases, dos diagramas de secuencia y ADR.	Misma solicitud de la iteración 2 (parte de la Etapa 8).	Diagrama de clases por capas (Aplicación, Dominio, Infraestructura), secuencia del check-in (flujo principal), secuencia de la cancelación (regla de 48 h, PIN y reembolso) y 5 ADR con contexto, alternativas, decisión, consecuencias y trazabilidad.	Errores detectados: el primer diagrama de clases (Mermaid) quedó ilegible y se rehízo en Graphviz; una primera versión sacó tres servicios del recuadro de Aplicación; los nombres de dos participantes de las secuencias no coincidían con el diagrama de clases (AutorizacionRepository, falta Habitacion) y se corrigieron.	Propuesta: conservar los diagramas y verificar que cada nombre coincida con el glosario y el código. Revisar que los ADR reflejen decisiones que el equipo realmente comparte.
10	Entregar las etapas 7 a 9 en formato Word, con los diagramas incluidos y sus archivos fuente.	«y darmelos en word».	Documento .docx con las secciones 12, 13 y 15 y un registro de ajustes, más un zip con las fuentes (.dot, .mmd) y las imágenes de los diagramas.	Errores detectados al revisar el PDF generado: columnas de las tablas con el mismo ancho, un diagrama separado de su título y filas partidas entre páginas; se corrigieron. Limitación: es un documento aparte, aún no integrado en el .docx original de la actividad.	Propuesta: integrar las secciones al documento original del equipo, subir las fuentes de los diagramas al repositorio (carpeta diagrams) y revisar el texto antes de entregar.

Cisco Confidential

15. Etapa 9 – Validar la propuesta

Recorrido de cada caso de uso sobre los diagramas. La columna «Dónde se valida» indica el componente responsable de cada regla.

Caso de uso	Componente que recibe	Dónde se valida cada regla	Objetos que cambian y persistencia	Evento
CU-01 Consultar disponibilidad y tarifa	ReservaService.consultarDisponibilidad → ConsultarDisponibilidadHandler (Query)	Capacidad y tipo (BR-01, BR-02) en el filtro de lectura. Tarifa y +35% (BR-03, BR-04) en PoliticaTarifaria.	Ninguno. Lectura con DTO por IDisponibilidadReadModel. Sin transacción de escritura.	—
CU-02 Confirmar / cancelar / no-show	ReservaService (confirmarReserva, cancelarReserva, gestionarNoShow)	Garantía 50% o tarjeta (BR-05) en Reserva.confirmar. Bloqueo sin sobreventa (BR-19) en ServicioDisponibilidad. Plazo de 48 h (BR-06) en IPoliticaCancelacion. No-show a las 23:59 (BR-07) vía IReloj. PIN para condonar (BR-18) en IAutorizacionRepository.	Reserva (estado, garantía). Un commit por caso de uso mediante IUnitOfWork. Reembolso por IGestorGarantias tras el commit.	ReservaConfirmada, ReservaCancelada, NoShowDeclarado
CU-03 Check-in, early check-in, late check-out	CheckInService → RealizarCheckInHandler; CheckOutService.gestionarLateCheckOut	Identificación y garantía incidental (BR-11) en Estancia.iniciar. Habitación lista (NB-07) en el handler. Cargo 30% (BR-09, BR-10) en IPoliticaCargoHorario.	Reserva, Habitacion, Estancia y Folio en una sola transacción (ADR-002). Rollback si falla cualquier regla.	CheckInCompletado
CU-04 Salida, consumos, inspección y limpieza	CheckOutService.realizarCheckOut (nuevo, C-04); ConsumoService; LimpiezaService	Consumos con firma e inmediatos (BR-14) en Folio.agregarCargo. Inspección exprés antes del cierre (BR-15) en Estancia.cerrar. Limpieza 35–45 min y estado «Limpia/Inspeccionada» (BR-16) en Habitacion.marcarLimpia.	Folio (cargos y liquidación), Estancia (cierre) y Habitacion (Sucia → Limpia/Inspeccionada). Una transacción por operación.	ConsumoAgregado, CheckOutRealizado, HabitacionLimpiaInspeccionada

Validación	Pregunta	Resultado propuesto (revisar por el equipo)
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	Parcial. Cada componente traza a NB/BR/US/CU (ver ADR y recorrido). Falta historia para check-out y reportes (H-U2-01) y está pendiente la matriz de trazabilidad formal (entregable 04).
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	Cumple con un ajuste: usar GarantiaReserva (anticipo/tarjeta) y GarantiaIncidental por separado para no confundir «garantía». Agregar al glosario: Política tarifaria, Estancia como agregado y Disponibilidad (servicio de dominio).
Responsabilidad	¿Las reglas permanecen en el dominio o en el patrón elegido con justificación?	Cumple. Los cálculos de BR-04, BR-06, BR-09, BR-10 y BR-13 viven en el dominio y las políticas (H-02, H-03). La validación del PIN vive en aplicación, con justificación.
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	Cumple. Todos los participantes de las dos secuencias existen en el diagrama de clases con el mismo nombre (CheckInService, RealizarCheckInHandler, ReservaService, Reserva, Estancia, Habitacion, IReservaRepository, IHabitacionRepository, IAutorizacionRepository, IGestorGarantias, IPoliticaCancelacion, IPoliticaCargoHorario, IUnitOfWork).

Cisco Confidential

Validación	Pregunta	Resultado propuesto (revisar por el equipo)
Persistencia	¿El dominio evita depender de SQL, ORM o infraestructura?	Cumple. El dominio solo conoce interfaces. El SQL directo existe únicamente en DisponibilidadReadRepository, detrás de IDisponibilidadReadModel (H-05, H-06).
Transacción	¿El límite de la operación y su manejo de error están definidos?	Cumple. Un Unit of Work por caso de uso, con commit único y rollback ante excepción de dominio. Los efectos externos van tras el commit (H-07). El check-in toca tres agregados en una transacción (ADR-002).
IA	¿Cada aporte de IA fue revisado y su uso está registrado?	Cada uso de la IA fue registrado dentro de la bitácora y se reviso que cumpliera con las instrucciones debidas.

16 Entregables

Entregable	Contenido minimo
01 Informe de arquitectura	Documento PDF con portada, alcance, comparación de patrones, mapa de dominio, Service Layer, Repository, Unit of Work, CQRS, revisión SOLID, ADR y conclusiones.
02 Diagrama de dominio y clases	Archivo fuente y exportación legible en PDF o PNG.
03 Diagramas de secuencia	Dos archivos fuente y sus exportaciones.
04 Matriz de trazabilidad	Relación entre necesidades, reglas, historias, casos de uso, componentes y decisiones.
05 Bitacora de IA	Prompts, respuestas relevantes, errores detectados, correcciones y decisiones humanas.
06 Codigo base	Estructura de carpetas y código mínimo que represente la arquitectura propuesta.
07 Repositorio GitHub	Todos los archivos organizados, historial de commits y README con instrucciones de revisión.
08 Presentacion y defensa	Exposición breve donde todos los integrantes puedan explicar y defender decisiones.

Estructura sugerida del repositorio

- docs Informe, ADR, trazabilidad y bitacora de IA
- diagrams Archivos fuente y exportaciones
- src Dominio, aplicacion e infraestructura
- tests Escenarios o pruebas seleccionadas
- README Empresa, alcance, instrucciones y enlaces a entregables

Cisco Confidential

17 Rubrica de evaluacion

Criterio	Evidencia esperada	Puntos
Continuidad y trazabilidad	Usa los productos de U2, conserva el alcance y relaciona decisiones con evidencia.	15
Seleccion del patron	Compara alternativas y justifica la eleccion con criterios concretos del caso.	15
Mapa de dominio	Distingue responsabilidades, limites, lenguaje, invariantes y eventos de forma coherente.	20
Service Layer y persistencia	Coordina casos de uso, separa reglas y define Repository y Unit of Work.	15
CQRS y SOLID	Separa operaciones con criterio y revisa el diseño mediante evidencia concreta.	10
UML y ADR	Los diagramas son consistentes y las decisiones registran contexto, alternativas y consecuencias.	15
Uso responsable de IA	Presenta iteraciones, detecta errores y explica las decisiones humanas.	5
Repositorio y defensa	Entrega organizada, legible y defendida por todos los integrantes.	5

Total 100 puntos

Cisco Confidential

18 Criterios de integridad academica

- La IA puede proponer, revisar y explicar, pero no reemplaza la justificación del equipo.
- Toda afirmación sobre el negocio debe provenir del caso o identificarse como supuesto pendiente de validación.
- El equipo debe conservar la autoría de sus decisiones y poder explicarlas sin leer una respuesta generada.
- El código, los diagramas o los textos generados deben revisarse antes de integrarlos al repositorio.
- Copiar la arquitectura de otro equipo o entregar contenido que no corresponde a la empresa asignada se considera incumplimiento.

19 Lista final de comprobacion

- [Si] Trabajamos con la misma empresa y el mismo alcance de la Unidad 2.
- [Si] La selección del patrón está respaldada por evidencia del caso.
- [Si] El mapa de dominio usa el lenguaje ubicuo del proyecto.
- [Si] Cada Application Service representa un caso de uso y coordina sin concentrar todas las reglas.
- [Si] Las interfaces de Repository hablan en términos del dominio.
- [Si] Los Commands y Queries tienen responsabilidades distintas y justificadas.
- [Si] Revisamos el diseño con SOLID y corregimos los hallazgos relevantes.
- [Si] Los diagramas de clase y secuencia son consistentes.
- [Si] Incluimos al menos tres ADR y una matriz de trazabilidad actualizada.
- [Si] La bitácora demuestra cómo usamos y validamos la IA.
- [Si] El repositorio contiene archivos fuente, exportaciones y un README claro.
- [Si] Todos los integrantes pueden defender las decisiones tomadas.

Cisco Confidential

20 Preguntas para la defensa

- ¿Qué evidencia del negocio influyó más en la selección del patrón?

La evidencia decisiva fue la presencia de reglas condicionales dependientes de tiempo, estado y múltiples umbrales cambiantes, sumadas a la necesidad de proteger invariantes estrictas.

- ¿Qué responsabilidad fue la más difícil de ubicar y cómo tomaron la decisión?

La responsabilidad más difícil de ubicar fue la validación de disponibilidad y prevención de sobreventa (overbooking cero - BR-19).

El dilema inicial: En la primera versión, asignamos el control del inventario global al agregado ReservaAggregate. Sin embargo, esto violaba el principio de responsabilidad única (SRP - Hallazgo H-01), pues una sola reserva no debe coordinar la disponibilidad de todas las demás habitaciones del hotel.

La decisión: Separamos las responsabilidades. La entidad Reserva únicamente protege su propio estado, sus fechas y su garantía. La validación de disponibilidad global la delegamos a un **Domain Service** especializado (ServicioDisponibilidad) y a la interfaz de lectura optimizada (IDisponibilidadReadModel).

- ¿Qué regla protege el diseño y qué ocurriría si estuviera en una capa incorrecta?

Un ejemplo claro es la regla BR-16 / NB-07 (Control de asignación de habitación hasta contar con estado Limpia/Inspeccionada).

Dónde está en nuestro diseño: Vive en el Dominio dentro del método ocupar() de la entidad Habitacion y es coordinada por el manejador del comando RealizarCheckInHandler.

Qué ocurriría si estuviera en una capa incorrecta:

Si estuviera en la interfaz de usuario / controlador: Un desarrollador futuro podría crear un nuevo canal de check-in (ej. tótem auto-servicio o App móvil) y olvidar incluir la condición if (estado == "Limpia"), permitiendo asignar habitaciones sucias a huéspedes.

Si estuviera en la base de datos (Stored Procedures/Triggers): Acoplaría la lógica de negocio al motor SQL, haciendo imposible probar el flujo con pruebas unitarias rápidas y desacopladas.

- ¿Qué cambió después de la revisión con SOLID?

La revisión SOLID (Etapa 7) nos llevó a realizar cuatro correcciones fundamentales en la arquitectura:

Patrón Strategy para reglas cambiantes (OCP - Hallazgo H-03): Reemplazamos los if/else de recargos y penalizaciones por interfaces de políticas (IPoliticaCancelacion, IPoliticaCargoHorario, IPoliticaSalidaAnticipada).

Segregación de Interfaces en Repositorios (ISP - Hallazgo H-05): Dividimos la interfaz pesada de IHabitacionRepository. Dejamos métodos de modificación en el repositorio de dominio y pasamos las consultas de solo lectura a IDisponibilidadReadModel.

Cisco Confidential

Inversión de Dependencias mediante Puertos (DIP - Hallazgo H-06): Desacoplamos la aplicación de la base de datos SQL y de servicios externos definiendo puertos como IGestorGarantias, IReloj y IAutorizacionRepository.

Manejo transaccional en eventos externos (Hallazgo H-07): Establecimos que los cobros o reembolsos bancarios externos se ejecutan únicamente después del commit de la base de datos a partir de eventos de dominio (ReservaCancelada).

- ¿Qué propuesta de la IA rechazaron o corrigieron y por qué?

  Rechazamos y corregimos dos propuestas principales generadas por la IA:

  1. **Modelos de lectura/escritura unificados en CQRS:** Inicialmente, la IA sugirió reutilizar las mismas entidades de dominio para responder las consultas de reportes (como ocupación diaria o disponibilidad). La rechazamos porque cargar agregados completos para un reporte administrativo satura la memoria y rompe el principio de CQRS. Definimos DTOs planos (HabitacionDisponibleDTO, OcupacionDiariaViewModel).

  2. **Uso de bases de datos físicamente separadas:** La IA propuso implementar sincronización asíncrona mediante Event Sourcing y dos bases de datos distintas para CQRS. La rechazamos porque la escala operativa de un hotel de 120 habitaciones no justifica la complejidad de sincronizar dos bases de datos; la separación a nivel de código (Commands vs Queries sobre la misma base de datos) es suficiente.

- ¿Qué consecuencia negativa aceptaron al tomar su decisión arquitectural principal?

Al adoptar Domain Model, Service Layer y Hexagonal/Puertos y Adaptadores:

Aceptamos la sobrecarga de diseño inicial y la proliferación de clases / abstracciones: En lugar de tener scripts simples de 50 líneas, creamos entidades, Value Objects, interfaces de políticas, Handlers, DTOs y repositorios.

Aceptamos transacciones que tocan múltiples agregados en el Check-in: En el caso de uso realizarCheckIn, la transacción (Unit of Work) modifica de forma atómica la Reserva, crea la Estancia y actualiza la Habitacion. Asumimos este compromiso en la capa de aplicación para mantener la consistencia inmediata exigida por el negocio.

Repositorio de github:

https://github.com/rulocm07/Equipo-9-Componentes-de-Logica.git

Cisco Confidential